

1. ¿Cuál es la diferencia entre monitoreo, observabilidad y alertamiento?

El monitoreo consiste en recolectar y visualizar métricas predefinidas de un sistema para saber su estado en tiempo real. La observabilidad va más allá: es la capacidad de entender el comportamiento interno de un sistema a partir de sus salidas externas (métricas, logs y trazas), permitiendo diagnosticar problemas no anticipados. El alertamiento es el mecanismo que notifica automáticamente cuando una métrica supera un umbral crítico, permitiendo reaccionar antes de que el problema escale.

2. ¿Por qué Prometheus trabaja bajo un modelo de scraping?

Prometheus adopta el modelo pull (scraping) porque ofrece mayor control: es Prometheus quien decide cuándo y con qué frecuencia consultar cada servicio. Esto facilita detectar si un servicio dejó de responder, evita que los servicios necesiten conocer la dirección del servidor de monitoreo, y centraliza la configuración de intervalos de recolección. Además, simplifica la seguridad porque solo Prometheus inicia las conexiones.

3. ¿Qué es un exporter y por qué se necesita?

Un exporter es un proceso intermediario que traduce las métricas de un sistema o aplicación al formato que Prometheus puede consumir. Se necesita porque muchos servicios (MariaDB, el sistema operativo, Docker) no exponen métricas en formato Prometheus de forma nativa. El exporter consulta al servicio, transforma los datos y los expone en un endpoint `/metrics` que Prometheus puede scrapear.

4. ¿Qué diferencia hay entre monitorear el host y monitorear contenedores?

Monitorear el host significa observar los recursos físicos o de la VM: CPU total, RAM total, disco, red y carga del sistema operativo, usando herramientas como `node_exporter`. Monitorear contenedores significa observar el consumo individual de cada contenedor en ejecución: cuánta CPU usa cada uno, cuánta memoria, su estado, reinicios y tráfico de red, usando herramientas como `cAdvisor`. Un contenedor puede consumir recursos excesivos sin que el host parezca saturado visto de forma general, por eso ambos niveles son necesarios.

5. ¿Qué métricas son más importantes para detectar saturación del servidor?

Las métricas clave son el porcentaje de uso de CPU, el porcentaje de memoria RAM utilizada, el espacio disponible en disco por partición, el `load average` del sistema (`node_load1`, `node_load5`, `node_load15`) y el tráfico de red entrante y saliente. Juntas permiten identificar cuellos de botella antes de que el servidor deje de responder.

6. ¿Qué métricas son relevantes para una aplicación Django?

Las métricas más importantes son el número de requests HTTP por segundo, la distribución de códigos de respuesta (2xx, 4xx, 5xx), la latencia de las peticiones (tiempo de respuesta), el número de errores, las consultas a base de datos por request y la disponibilidad del endpoint `/metrics`. Estas métricas permiten detectar degradación de rendimiento y errores en la aplicación antes de que los usuarios lo reporten.

7. ¿Qué métricas son relevantes para Tomcat/JVM?

Las métricas clave son el uso de memoria heap y non-heap de la JVM, el número y duración de los ciclos de Garbage Collection, los threads activos y en espera, el número de requests procesados por Tomcat, los errores HTTP generados y el uptime del proceso.

Estas métricas permiten detectar memory leaks, saturación de threads y degradación del rendimiento de la aplicación Java.

8. ¿Qué riesgos existen al exponer Prometheus, Grafana o exporters públicamente?

Exponer estos servicios públicamente sin protección permite que cualquier persona acceda a métricas sensibles del sistema como estructura de red, nombres de contenedores, versiones de software y patrones de tráfico, información útil para un atacante. Grafana sin autenticación fuerte puede dar acceso a dashboards completos. Los exporters sin restricción pueden revelar detalles de la base de datos, del host y de las aplicaciones. Lo correcto es restringirlos a red interna y acceder mediante VPN o proxy inverso con autenticación.

9. ¿Por qué no deben guardarse contraseñas SMTP en el repositorio?

Porque los repositorios, especialmente los públicos, son accesibles por cualquier persona. Una contraseña SMTP expuesta permite que terceros envíen correos en nombre de la cuenta, la usen para spam o phishing, y comprometan la cuenta de correo completa. Incluso en repositorios privados es mala práctica porque el historial de Git conserva los valores aunque se editen después. La solución correcta es usar variables de entorno o archivos `.env` excluidos del repositorio mediante `.gitignore`.

10. ¿Qué mejoras implementarían si esta solución se llevara a producción?

Se implementaría almacenamiento persistente y con retención configurada para Prometheus, se usaría un sistema de gestión de secretos como HashiCorp Vault o AWS Secrets Manager para las credenciales, se protegerían todos los servicios de monitoreo detrás de un proxy inverso con HTTPS y autenticación. Se agregarían trazas distribuidas con herramientas como Jaeger o Tempo, se centralizarían los logs con Loki o Elasticsearch, se configurarían alertas más granulares con escalado por severidad y canales múltiples (Slack, PagerDuty), y se automatizaría el aprovisionamiento de dashboards y alertas mediante código (Grafana provisioning).